



Politecnico  
di Torino

# Parallelizing Minimax



---

PRESENTED BY DAVIDE FASSIO (S323584)

# Minimax



# Negamax

**Purpose:** Evaluates the score in a zero-sum game after a set number of moves, assuming optimal play by both players.

**Concept:**

- Max player (first mover) aims to maximize the score.
- Min player (opponent) aims to minimize the score.

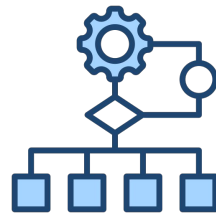
Simplified version of Minimax using symmetry in zero-sum games.

**Key Idea:**

- The evaluation of a position is the negation of the opponent's evaluation.
- One's score is  $+x$ , the opponent's score is  $-x$ .

**Advantage:**

- Easier implementation by treating both players symmetrically.



# Alpha-Beta Pruning

---

**What?** A search optimization technique applied to the Minimax algorithm.

- **Alpha** and **Beta**: the best scores the maximizer/minimizer can guarantee
- Skips evaluating branches of the game tree when it's clear they won't affect the final decision.

**Why?**

- **Efficiency**: Reduces the number of game states explored. Time savings can be invested to explore more levels of the game tree, leading to stronger play.
- **Same Decision**: Produces the same result, as pruned branches have no impact on the best move.

**Consequence**: The search phase in subtrees is sequential and can no longer be easily parallelized.

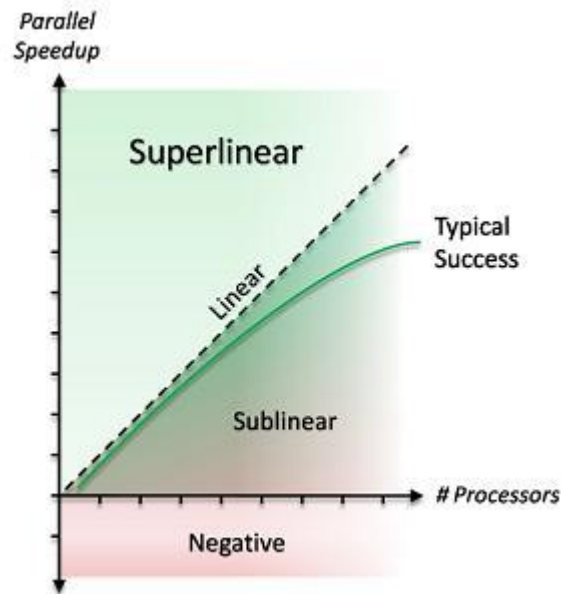
# Parallelization

## Why Parallelize Minimax while using Alpha-Beta pruning?

- Alpha-Beta drastically reduces the search space in Minimax.
- Modern processors emphasize parallelism (more cores, not higher clock speeds), needing us to leverage concurrent search.

## Key Concepts: Scalability vs Speedup

- **Scalability (Nodes per Second):** how well the algorithm's NPS increases with additional processing cores. High scalability means the algorithm efficiently exploits more cores to search more nodes.
- **Speedup (Time to Depth):** Reflects the reduction in total search time to reach a certain depth. A true speedup appears when adding processors actually shortens the time it takes to complete the same search.



# Main Algorithms

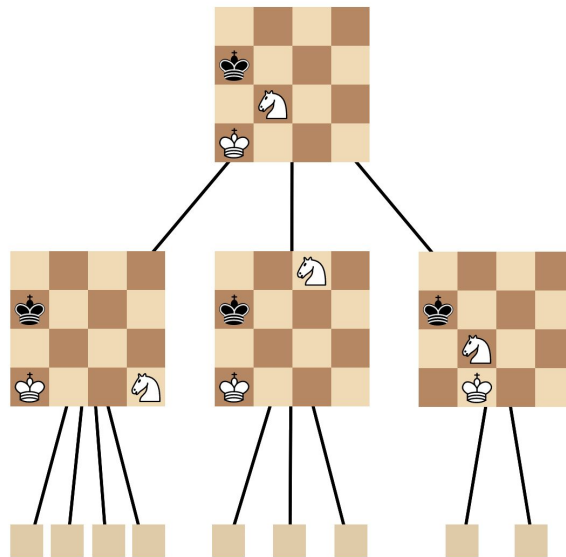
---

## Shared Hash Table

- Lazy SMP
- ABDADA

## Parallel Alpha-Beta

- Principal Variation Splitting
- Young Brothers Wait Concept
- Dynamic Tree Splitting



# Shared Hash Table

---

**What?** A shared hash table is a hash-based data structure accessible by multiple processes or threads, often allocated in shared memory for concurrent use.

|      | Locks                                                                                                                                                                                        | Lock-Free                                                                                                                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pro  | <ul style="list-style-type: none"><li>Relies on well-understood <u>locks primitives</u> (e.g., mutexes).</li><li><u>Simpler</u> to implement and maintain.</li></ul>                         | <ul style="list-style-type: none"><li>Relies on <u>atomic operations</u> (e.g., compare-and-swap) for concurrency.</li><li>Minimizes blocking , so it <u>scales better</u> under high concurrency.</li></ul> |
| Cons | <ul style="list-style-type: none"><li>Contention when multiple threads attempt to acquire the same lock.</li><li>Potential deadlocks and priority inversions if locks are misused.</li></ul> | <ul style="list-style-type: none"><li>Significantly <u>more complex</u> to design, implement, and verify.</li></ul>                                                                                          |

# Lazy SMP

---

**Lazy SMP** is a parallel search approach that leverages a shared transposition table among multiple search instances. Each instance explores the same position but may use different depth settings or root move orderings, thus introducing useful nondeterminism, otherwise depending on random timing fluctuations.

In practice, Lazy SMP is remarkably straightforward to implement and often scales well up to eight or more cores, boosting playing strength.

Implemented by:

- Stockfish (since Stockfish 7, 2016) and its forks (Berserk, Koivisto, ...)
- Ethereal (since Ethereal 8.60 SMP, 2017)
- Dragon by Komodo

# ABDADA

---

**Name:** Alpha-Bêta Distribué avec Droit d'Aînesse = Distributed Alpha-Beta Search with Eldest Son Right

## Key Concepts:

- Shared Hash Table: Approach allowing all processors to start searching from the root simultaneously.
- Young Brothers Wait Concept: Parallel evaluation is delayed until the “eldest son” node is fully evaluated.

## Features:

- Introduction of additional transposition table information (e.g., the number of processors searching each node) to efficiently manage speculative parallelism.
- Simple serial-style recursive structure.

# Parallel Alpha-Beta

---

## Overview

- The Alpha-Beta tree is subdivided so that different processors handle different subtrees.
- Because Alpha-Beta search is fundamentally a serial algorithm, distributing the tree to multiple processors introduces significant complexity.

**Key Benefits:** Greater scalability and faster decision-making in highly branching game trees.

## Challenges

- Coordinating shared bounds (alpha and beta) across processors can be difficult.
- Synchronization and communication overhead must be carefully managed to realize speedups.

# Principal Variation Splitting

---

**What?** Parallelizing Principal Variation Search

**How?**

- First, pick your best guess for the move in a position and search it with a full alpha-beta window to get an exact score.
- For each of the other moves, do a null window search to quickly see if it “fails high” (better than current best) or “fails low.”

**Why?** Efficiency

- Null window searches are fast: once you detect a score outside the narrow window, you stop early.
- Most moves fail low (are worse than the best guess), so you quickly confirm they’re not improvements without a full search.

**Parallelization:**

- After searching the first (best) move, remaining moves can be searched in parallel with a null window.
- If any move fails high, you must re-search it with a full window.

# Young Brother Wait Concept

---

**Why?** Increase efficiency of parallel alpha-beta search in game-tree exploration.

**How?**

1. Oldest Brother First
  - Full computational resources focus on evaluating the first (oldest) child node.
  - If this node triggers a beta cutoff (or significantly raises alpha), large sections of the search space can be pruned early.
2. Wait Mechanism
  - Younger siblings wait while the oldest brother searches.
  - If no cutoff occurs, the younger brothers begin their parallel searches.
3. Efficiency Gains
  - Prevents unnecessary work on siblings that might be pruned.
  - Improves cost-effectiveness in distributed or parallel search environments.

**Impact:** fundamental strategy applied in many other algorithms and contexts.

# Dynamic Tree Splitting

---

**Why?** Fully parallel game-tree search with continuous workload distribution (no idle processors when the search tree is busy)

**How?**

- Each busy processor publishes its current subtree state (position, bounds, moves, repetition info) in a “split block” in shared memory
- Idle processors pick up a subtree from these published states to work on, eliminating downtime

**Implementation:** Used in the Cray Blitz, by Robert Hyatt, on a 16-processor Cray C916/1024 (with 8 GiB shared memory)

# Conclusion

---

- There are **numerous research ideas** for parallelizing Minimax with Alpha-Beta pruning.
- Despite the variety of methods, currently, **Lazy SMP dominates in practice**.
- **Scalability Limitations:** Lazy SMP is effective only up to about 8–12 cores. This poses a major bottleneck for processors with higher core counts (e.g., AMD Threadripper) or multi-processor servers.
- **Unresolved Problem:** We still lack a practical algorithm that can fully utilize the power of highly parallel architectures.

Sources: Chess Programming Wiki

**Thanks for your  
attention!**

---